

Problem1: Cache (31 points)

Consider a **16-bit machine** with a **2-way** set associative cache. The size of each block is **8 bytes**, and the total number of sets is 2. **LRU** policy is used for eviction and **write back** policy is used. The following table shows the content of the data cache at time T.

Set	Tag	Valid	Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0	0x0C0	1	0x2F	0x54	0x30	0xCC	0x66	0x9D	0xBC	0x7B
0	0xF0E	0	0xEE	0xD6	0xAC	0xAC	0xD0	0x0A	0xAC	0x9F
1	0x140	0	0x4C	0xAB	0xCD	0xEF	0xAD	0xBE	0xFF	0xEF
1	0x1E0	1	0x4B	0x19	0x24	0x25	0x99	0xDD	0x24	0xAF

- What is the total size of the cache? (3')
- How would a 16-bit physical memory address be split into tag/set-index/block-offset fields in this machine? (6')
tag [1] bits, set-index [2] bits, and block-offset [3] bits
- A short program will read memory in the following sequences starting from time T. Each access will read **one byte**. Please fill the following blanks. If there is a cache miss, fill '-' for 'Byte Returned'. (10')

Order	Address	Hit/Miss	Byte Returned
1	0xF0E2	[1]	[2]
2	0x0C08	[3]	[4]
3	0x0C03	[5]	[6]
4	0x2C3B	[7]	[8]
5	0x1E0A	[9]	[10]

One of the TA writes a program and tests it on this cache. The size of short type is **2 bytes**. The cache is **empty** before the execution of the program. Please only consider **data cache** access and ignore instruction cache access. Other processes will not interfere the status of memory and cache during this execution.

```
1. #define N 8
2. short A[N];
3. short B[N];
4. short C[N];
5.
6. void dot_product() {
7.     int i;
8.     for (i = 0; i < N; i++) {
9.         C[i] = A[i] * B[i] + C[i];
10.    }
11. }
```

Assume that the address of A[0] is 0xa00. The address of B[0] is 0xb00. The address of C[0] is 0xc00. Answer the following questions:

4. Please calculate **the cache miss rate**. (3')

5. We can modify the cache through one of the following ways:

- a) Double the number of sets while keeping the cache size unchanged.
- b) Double the block size while keeping the cache size unchanged.
- c) Increase the cache size via doubling the number of cache lines in each set.

Please choose the option(s) above that can optimize the miss rate of the original program and calculate the miss rate respectively.(6')

6. Besides modifying the cache hardware parameters, can you propose a software-based solution to improve the cache hit rate? (3')

(Hint: Consider adjusting the starting addresses of arrays.)

Problem 2: Signal & I/O (30 points)

The program below is a parent that opens a log file, forks 3 children that each do a short job, and uses a SIGCHLD handler to reap them. Assume the program is launched from a shell as `./logger`, with the original stdout connected to a terminal.

```
1  #include "csapp.h"
2  volatile sig_atomic_t reaped = 0;
3  int logfd;                      /* set by main before any fork */
4
5  void chld_handler(int sig) {
6      pid_t pid;
7      while ((pid = waitpid(-1, NULL, WNOHANG)) > 0) {
8          printf("reaped pid %d (total=%d)\n",
9                 (int)pid, (int)(reaped + 1));
10         reaped++;
11     }
12 }
13
14 int main(void) {
15     sigset_t mask, prev;
16     Sigemptyset(&mask);
17     Sigaddset(&mask, SIGCHLD);
18
19     logfd = Open("/tmp/run.log",
20                  O_WRONLY | O_CREAT | O_APPEND, 0644);
21     Dup2(logfd, STDOUT_FILENO); /* fd 1 now -> /tmp/run.log */
22
23     Signal(SIGCHLD, chld_handler);
24     Sigprocmask(SIG_BLOCK, &mask, &prev);
25
26     for (int i = 0; i < 3; i++) {
27         if (Fork() == 0) {
28             printf("child %d starting\n", i);
29             exit(0);          /* flushes stdio buffer */
30         }
31     }
32
33     printf("parent: forked 3 children\n");
34     Sigprocmask(SIG_SETMASK, &prev, NULL);
35
36     while (reaped < 3)
37         Pause();
```

```
38
39     printf("parent: all reaped, exiting\n");
40     return 0;
41 }
```

1. The three-table model — what is fd 1? (6 pts)

a. Immediately after line 21 finishes (so `Dup2(logfd, STDOUT_FILENO)` has just returned, but the first `Fork()` has not yet been called), what is the reference count of the system-wide open file table (OFT) entry that backs `/tmp/run.log`? List every descriptor-table slot that points at that entry. (2 pts)

b. After all three `Fork()` calls on line 27 have returned in the parent (but before any child has executed `exit(0)`), what is the reference count of the same OFT entry? List every descriptor-table slot, across all four processes, that points at it. (2 pts)

c. The terminal (the shell's original stdout) sees **zero bytes** from this program. Why? Answer in one sentence using the three-table model. (2 pts)

2. Race condition — find the hang, then fix it (14 pts)

a. There is a race between lines 36 and 37 that can make the program hang forever. Describe a concrete interleaving of events (parent ↔ kernel ↔ children) that produces the hang. Reference the value of `reaped` at each step and the state of the `SIGCHLD` pending bit. (4 pts)

b. Rewrite lines 34, 36, and 37 so that the hang is impossible. You may use at most one additional local variable. Show full code with line numbers — your rewrite must use `sigsuspend` and must keep `SIGCHLD` blocked everywhere except atomically inside the `wait`. (4 pts)

c. Explain in 2–3 sentences why `sigsuspend(&prev)` closes the race that if `(reaped < 3) Pause();` leaves open. Your explanation must reference what `sigsuspend` does to the process mask before, during, and after the call. (3 pts)

d. Suppose a colleague proposes a different fix: replace line 37 with `usleep(100);` and loop until `reaped == 3`. Give one concrete reason this is strictly worse than the `sigsuspend` fix. (3 pts)

3. Async-signal safety + lost bytes (10 pts)

a. The handler on lines 5–12 violates at least two of the G0 / G1 / G2 safety guidelines from the lecture. Name each violation explicitly (which guideline, what line, what specifically is wrong). (4 pts)

b. Rewrite the body of `chld_handler` (lines 6–11) to comply with all three guidelines. Use `sio_puts/sio_putl` (and not `printf/snprintf`). Your rewrite must explicitly save and restore `errno`. (4 pts)

c. Even after your rewrite from (b), there is a subtle interaction with the `O_APPEND` flag on `/tmp/run.log` and the fact that `fd 1` has been `dup2'd` to it. State whether each handler-side `sio_puts` write can be safely interleaved with a write from main's `printf` on line 33 without bytes from the two writes being intermixed mid-line. Justify in one sentence, citing exactly one I/O concept from week 8. (2 pts)

Problem 3: Virtual Memory (39 points)

The following system specifications apply to all sub-problems unless stated otherwise. These match a simplified x86-64 configuration from the lecture:

- Virtual addresses are **48 bits** wide.
- Physical addresses are **52 bits** wide.
- The page size is **4 KB** (2^{12} bytes).
- The system uses a **4-level** page table with **9 index bits per level**.
- Each page table entry (PTE) is **8 bytes**.
- The L1 data cache is **32 KB**, organized as **64 sets × 8-way × 64-byte lines**.

1. Bit-width fundamentals (8 pts)

Fill in the following table.

Quantity	Value
VPO / PPO bits	[a]
VPN bits	[b]
PPN bits	[c]
Entries per page-table page	[d]
Index bits per page-table level	[e]
Cache offset (CO) bits	[f]
Cache index (CI) bits	[g]
Cache tag (CT) bits (of physical address)	[h]

2. Page-table memory with regular and huge pages (15 pts)

A process maps a single **contiguous 8 MB** region starting at virtual address `0x0000_0000_0040_0000`. No other virtual pages are mapped. Each page-table page is 4 KB and holds 512 PTEs.

The 48-bit virtual address is decomposed as:

Field	L1 (PML4)	L2 (PDPT)	L3 (PD)	L4 (PT)	VPO
Bit range	47–39	38–30	29–21	20–12	11–0
Width	9 bits	9 bits	9 bits	9 bits	12 bits
What it selects	PML4 entry	PDPT page	PD page	PT entry	Byte within page

a. Using 4 KB regular pages, how many page-table pages are needed at each level? Fill in the table and give the total page-table memory in KB. Show which indices are populated at each level. (10 pts)

Hint: first compute the L1–L4 indices of the start address `0x0000_0000_0040_0000` and the end address `0x0000_0000_00BF_FFFF`, then determine how many distinct L3 and L4 entries are needed.

Level	Populated indices	Page-table pages
L1 (PML4)	[a1]	[a2]
L2 (PDPT)	[b1]	[b2]
L3 (PD)	[c1]	[c2]
L4 (PT)	[d1]	[d2]
Total		[e]

Total page-table memory: [f] KB

b. x86-64 supports 2 MB huge pages: a Level-3 (PD) entry can set the PS (Page Size) bit and directly map a 2 MB physical frame, eliminating the need for a Level-4 (PT) table beneath it. How many page-table pages are needed at each level for the same 8 MB mapping using 2 MB huge pages? How much page-table memory (in KB) is saved compared to part (a)? (4 pts)

c. In one sentence, explain why multi-level page tables save memory compared to a single flat page table for a typical process that uses only a small fraction of the 48-bit virtual address space. (1 pt)

3. VIPT cache design constraint (8 pts)

Modern processors use a **Virtually Indexed, Physically Tagged (VIPT)** cache to overlap the cache lookup with TLB translation. The key requirement is:

> The bits used for cache indexing must come entirely from the page offset, so that the **virtual** and **physical** cache indices are identical.

a. Using the L1 data cache specifications above (32 KB, 64 sets, 8-way, 64-byte lines), is VIPT feasible with 4 KB pages? Show the computation. (3 pts)

b. The hardware team proposes doubling the L1 cache to **64 KB**. Two designs are on the table:

- **Design A:** double the number of sets to 128 (128 sets $\times$ 8-way $\times$ 64-byte lines).
- **Design B:** double the associativity to 16-way (64 sets $\times$ 16-way $\times$ 64-byte lines).

For each design, state whether VIPT still works with 4 KB pages. Show the computation for each. If a design breaks VIPT, state the minimum page size that would restore the condition. (5 pts)

4. Efficiency of mmap for file copying (8 pts)

Consider the following two approaches to copying an n -byte file to stdout.

Approach 1 — `read` + `write`:

```
1 char buf[N];
2 read(fd, buf, N);           /* file -> user buffer */
3 write(STDOUT_FILENO, buf, N); /* user buffer -> stdout */
```

Approach 2 — `mmap` + `write`:

```
1 char *p = mmap(NULL, N, PROT_READ,
2               MAP_PRIVATE, fd, 0);
3 write(STDOUT_FILENO, p, N); /* mapped region -> stdout */
```

a. Which approach copies faster? And please describe each approach's copy process, the number of copies involved, and the memory regions involved to explain the reason (use the terms: disk, kernel page cache, user buffer, kernel output buffer). (5 pts)

b. Suppose the program needs to read only a small header (the first 4 KB) from a very large file (e.g., 1 GB). Explain in 2–3 sentences why mmap is more efficient than read(fd, buf, 1GB) followed by inspecting the first 4 KB of buf. (3 pts)